Streaming Community Calendar — Open Source Setup & Customization Guide

(NULL Baseline Deployment Manual)

1. Project Overview & Dual-Stack Architecture

The Streaming Community Calendar is a zero-dependency, self-hosted system built with a dual-stack design:

Dual-Stack Components

Component	Key Responsibilities
Node.js Backend (`server.js`)	Handles web server endpoints, REST API routes (`/api/events`, `/api/status/all`), live stream polling for Kick, Twitch, and YouTube, and dispatches automated Discord notifications.
Python Desktop Interface (`calendar.py`)	Runs a PyQt6/QtWebEngine desktop application that loads the interactive calendar interface (`calendar_index.html`) and connects local system telemetry via QWebChannel.

2. Setting Up Environment Variables (.env)

Create a `.env` file in the root directory by copying `.env.example`.

.env File Example

```shell
PORT=55565
NODE_ENV=production
DISCORD_WEBHOOK_URL=WEBHOOK_ENDPOINT_URL_HERE
YOUTUBE_API_KEY=YOUR_YOUTUBE_DATA_API_KEY
TWITCH_CLIENT_ID=YOUR_TWITCH_CLIENT_ID
TWITCH_CLIENT_SECRET=YOUR_TWITCH_CLIENT_SECRET
```

3. How to Customize & Add Your Own Streamer Info

A. Editing Platform Channel Polling Lists (server.js)

Open `server.js` and locate the channel array definitions near the top of the file:

1. **Kick Channels:** Add exact Kick usernames/slugs in `KICK_CHANNELS`:

```javascript
const KICK_CHANNELS = [
    "your_kick_username_1",
    "your_kick_username_2"
];
```

2. **Twitch Channels:** Add lowercase Twitch usernames in `TWITCH_CHANNELS`:

```javascript
const TWITCH_CHANNELS = [
    "your_twitch_username_1",
    "your_twitch_username_2"
];
```

3. **YouTube Channels:** Add channel display names and exact YouTube Channel IDs (starting with UC...) in `YOUTUBE_CHANNELS`:

```javascript
const YOUTUBE_CHANNELS = {
    "Creator Name 1": { "id": "UCXXXXXXXXXXXXXXXXXXXXXX" },
    "Creator Name 2": { "id": "UCYYYYYYYYYYYYYYYYYYYYYY" }
};
```

B. Customizing Weekly Schedules & Creator Profiles (calendar_index.html)

Open `calendar_index.html` (and `calendar.py`) to set up your community roster and daily schedule:

1. **Weekly Repeating Schedule (weeklySchedule):** Map days of the week (0 = Sunday, 1 = Monday, ..., 6 = Saturday) to array lists of events:

```javascript
const weeklySchedule = {
    0: [{ time: "12:00 PM", name: "StreamerName", note: "Community Stream" }],
    1: [{ time: "6:00 PM", name: "StreamerName2", note: "Weekly Event" }]
};
```

2. **Streamer Directory & Bios (streamerProfiles):** Add bios and social media chips for creators on your roster:

```javascript
const streamerProfiles = {
    "StreamerName": {
        bio: "Short bio describing the streamer's channel and content.",
        links: [
            { name: "Twitch", url: "STREAMER_TWITCH_URL" },
            { name: "Discord", url: "COMMUNITY_DISCORD_URL" }
        ]
    }
};
```

3. **Sidebar Banners & Links:** Locate `.esports-banner-box` in the HTML body to replace logos, community bios, and custom form links.

4. Running the Application

4. **Start the Backend Server:**

```shell
node server.js
```

5. **Launch the Desktop Application:**

```shell
python calendar.py
```

5. Deployment & Maintenance Checklist

4. Restart the Node backend whenever you modify `server.js` or `.env` so active background polling loops pick up updated arrays.
5. Verify that your GitHub repository contains only NULL placeholder files and `.env.example` before pushing code publicly.